Agentic Data Quality POC

Full Technical Documentation

Version	1.0
Last updated	2026-03-06
Repository scope	end-to-end pipeline, simulator, Databricks bundle, and AI agent app
Runtime target	Databricks Free Edition using catalog workspace and serverless-compatible patterns

Prepared as a professional reference document from the supplied source text.

Table of Contents 1
1. Purpose and Scope 2
2. High-Level Architecture 3
2.1 End-to-End Architecture Diagram 3
2.2 Runtime Technology Stack 4
3. Repository Structure and Responsibilities 4
4. Databricks Asset Bundle Design 4
4.1 Resources 4
4.2 Job Environment 5
4.3 Config Variables 5
5. Data Model and Contracts 5
6. Metadata-Driven Pattern (Core of this POC) 6
6.1 What “metadata-driven” means here 6
6.2 Metadata-Driven Execution Flow 6
6.3 Why this is valuable 6
6.4 Current POC Boundaries 7
7. Pipeline Internals (pipeline/poc_pipeline.py) 7
7.1 Entry and Dispatch 7
7.2 SetupMetadata Step 7
7.3 IngestBronze Step 8
7.4 Scd2Silver Step 8
7.5 DqEngine Step 9
7.6 DQ Operational Modes 9
8. Source Data Simulator (scripts/00_mongo_simulator.py) 9
9. App Design (app/agent_app.py) 9
9.1 Endpoints 9
9.2 SQL Access Model 10
9.3 Analysis Modes 10
9.4 LLM Provider Strategy 10
9.5 UI Rendering 10
10. Agentic AI Value in this POC 10
11. Configuration and Secrets Handling 11
11.1 .env and .load-env.ps1 11
11.2 Required Inputs 11

12. Design Patterns Used 11
13. Operational Runbook (Condensed) 12
14. Known Constraints and Tradeoffs 12
15. Extension Paths 12
16. Summary 12

1. Purpose and Scope

This document explains the full architecture and implementation of the project:

- metadata-driven ingestion and transformation in Databricks
- SCD Type 2 Silver model
- rule-based data quality engine
- issue logging model
- AI-assisted remediation agent (FastAPI app)
- deployment model using Databricks Asset Bundles

The target runtime is Databricks Free Edition, using catalog workspace and serverless-compatible patterns.

2. High-Level Architecture

2.1 End-to-End Architecture Diagram

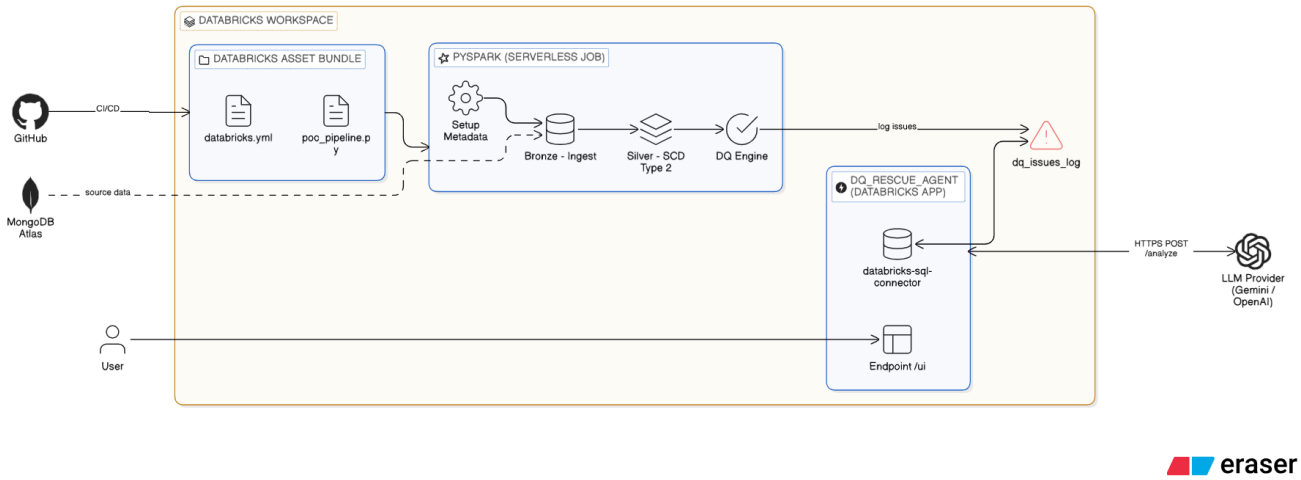

Figure 1. Improved rendering of the end-to-end architecture described in the source text.

Original text diagram preserved below for completeness:

```
+---------------------+          +---------------------------------------------+
| MongoDB Atlas       |          | Databricks Job: poc_pipeline_job            |
| ecommerce.customers |  ---->   |                                             |
+---------------------+          |  [1] SetupMetadata                           |
                                 |  [2] IngestBronze                            |
                                 |  [3] Scd2Silver                              |
                                 |  [4] DqEngine                                |
                                 +----------------------+----------------------+
                                                        |
                                                        v
                                 +---------------------------------------------+
                                 | Delta Tables in workspace.agentic_poc       |
                                 | - metadata_config                            |
                                 | - dq_rules                                   |
                                 | - bronze_customers                           |
                                 | - silver_customers                           |
                                 | - dq_issues_log                              |
                                 +----------------------+----------------------+
                                                        |
                                                        v
                                 +---------------------------------------------+
                                 | Databricks App: Data Quality Agent          |
                                 | FastAPI + SQL connector + LLM provider      |
                                 | Endpoints: /ui /latest-issue /issues-summary|
                                 | /analyze /docs                               |
                                 +---------------------------------------------+
```

2.2 Runtime Technology Stack

Layer	Technology
Source simulation	Python 3.12 script + pymongo
Orchestration	Databricks Asset Bundles (databricks.yml)
Data processing	PySpark + Delta Lake
Modeling	SCD Type 2 merge logic
Data quality	SQL-expression rule engine in Spark
Issue persistence	Delta table (dq_issues_log)
App/API	FastAPI + Uvicorn
Warehouse access	databricks-sql-connector
LLM providers	Gemini (google-genai + fallback google-generativeai), OpenAI, Anthropic
Environment injection	PowerShell loader (.load-env.ps1)

3. Repository Structure and Responsibilities

```
.
|-- databricks.yml                 # Bundle definition (job + app + variables)
|-- pipeline/poc_pipeline.py       # Full pipeline implementation
|-- scripts/00_mongo_simulator.py  # Seed/add_good/add_bad source records
|-- app/agent_app.py               # FastAPI app, UI, SQL queries, LLM analysis
|-- app/requirements.txt           # App dependencies
|-- .load-env.ps1                  # Loads .env + BUNDLE_VAR_* vars
|-- .env.example                   # Required configuration keys
|-- README.md                      # User-oriented instructions
|-- commands_guide.txt             # Command-oriented runbook
```

4. Databricks Asset Bundle Design

File: databricks.yml

4.1 Resources

1. Job resource: poc_pipeline_job
2. App resource: dq_rescue_agent
   - Task SetupMetadata
   - Task IngestBronze depends on SetupMetadata
   - Task Scd2Silver depends on IngestBronze
   - Task DqEngine depends on Scd2Silver
   - command: uvicorn agent_app:app --host 0.0.0.0 --port 8000
   - env vars injected from bundle variables

4.2 Job Environment

- environment_version: "5"
- dependencies for job task runtime:
- pymongo==4.16.0
- dnspython==2.8.0

4.3 Config Variables

Main variables:

- mongodb_uri
- llm_provider
- gemini_api_key
- dq_fail_on_issues (default false)
- databricks_server_hostname
- databricks_http_path
- databricks_token

dq_fail_on_issues is deployment-scoped: changing it requires bundle deploy.

5. Data Model and Contracts

Namespace:

- catalog/schema namespace: workspace.agentic_poc

Tables:

1. metadata_config
2. dq_rules
3. bronze_customers
4. silver_customers (SCD2)
5. dq_issues_log

metadata_config: source-level ingestion config

- key fields: source_name, source_format, source_options, target_table, is_active

dq_rules: active rule definitions

- key fields: rule_id, rule_sql, severity, is_active

bronze_customers: normalized records from source

- includes source_document_id, ingested_at

silver_customers (SCD2): includes is_current, valid_from, valid_to, record_hash

dq_issues_log: one row per failed record per rule

- includes serialized record_payload, rule_id, timestamp

6. Metadata-Driven Pattern (Core of this POC)

6.1 What "metadata-driven" means here

The pipeline does not hardcode source connection options in the ingest step. Instead, SetupMetadata writes a metadata row into metadata_config, and IngestBronze reads that active row at runtime.

Operationally:

- control plane = metadata + rules tables
- execution plane = Spark steps consuming control metadata

6.2 Metadata-Driven Execution Flow

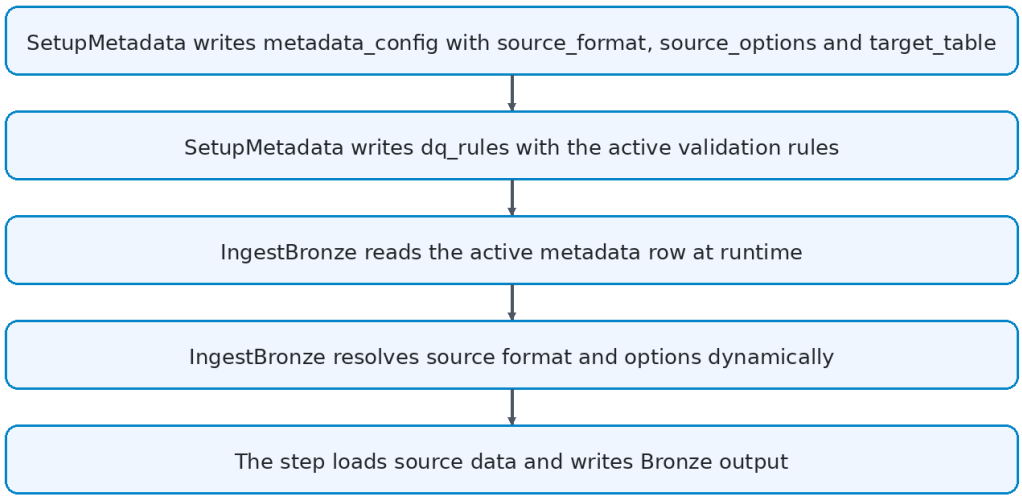

Figure 2. Improved metadata-driven execution flow.

```
SetupMetadata
  -> write metadata_config (source_format, source_options, target_table)
  -> write dq_rules

IngestBronze
  -> read active row from metadata_config
  -> resolve source format/options dynamically
  -> load data and write Bronze
```

6.3 Why this is valuable

- decouples pipeline logic from environment-specific source details
- supports controlled source changes without code rewrite
- enables reusable ingestion step for additional sources
- centralizes rule governance in dq_rules

6.4 Current POC Boundaries

- single active source: customers_mongo
- single domain table pair: bronze_customers -> silver_customers
- rule execution currently tied to current Silver records

7. Pipeline Internals (pipeline/poc_pipeline.py)

7.1 Entry and Dispatch

The script accepts --key value pairs and dispatches by --step.

Supported steps:

- SetupMetadata
- IngestBronze
- Scd2Silver
- DqEngine

Dispatch Flow

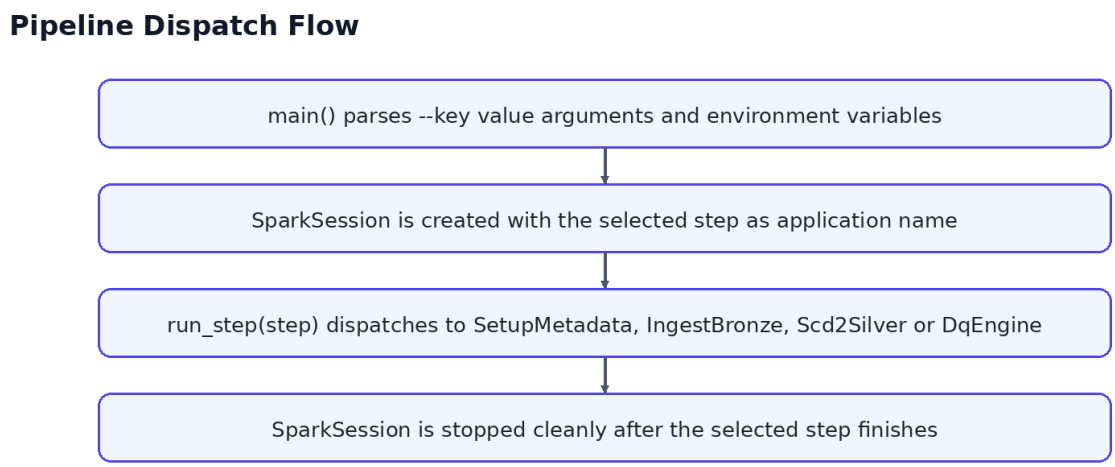

Figure 3. Improved dispatch flow for the pipeline entrypoint.

```
main()
  -> parse args/env
  -> SparkSession(appName=step)
  -> run_step(step)
      -> SetupMetadata | IngestBronze | Scd2Silver | DqEngine
  -> stop SparkSession
```

7.2 SetupMetadata Step

Responsibilities:

- create schema if needed
- overwrite metadata_config
- overwrite dq_rules
- create dq_issues_log if absent

Flow:

```
validate mongodb_uri
  -> CREATE SCHEMA IF NOT EXISTS
  -> WRITE metadata_config (overwrite)
  -> WRITE dq_rules (overwrite)
  -> CREATE dq_issues_log IF NOT EXISTS
```

7.3 IngestBronze Step

Responsibilities:

- load active source configuration from metadata
- read source records from MongoDB (via pymongo)
- normalize data types and required columns
- append into Bronze Delta table

Normalization contract:

- cast to deterministic types
- fallback updated_at to current timestamp
- map _id to source_document_id
- filter out null customer_id

Flow:

```
read metadata_config active row
  -> resolve effective mongo uri
  -> load source docs
  -> normalize_source()
  -> append to bronze_customers
```

7.4 Scd2Silver Step

Responsibilities:

- create Silver table if missing
- deduplicate latest Bronze row per customer_id
- compute record_hash for change detection
- close current records when changes detected
- insert new current versions

Flow:

```
ensure silver table exists
  -> read bronze
  -> latest record per customer_id
  -> compare hash vs current silver
     -> close changed current rows (is_current=false, valid_to=merge_ts)
```

```
     -> insert new current rows (is_current=true, valid_from=merge_ts)
```

7.5 DqEngine Step

Responsibilities:

- read active DQ rules from dq_rules
- evaluate each rule against current Silver records
- collect failed rows per rule
- append failures to dq_issues_log
- optionally fail task if dq_fail_on_issues=true

Flow:

```
load active silver + active rules
  -> for each rule:
       filter rows violating rule_sql
       build issue rows with payload
  -> append issues to dq_issues_log
  -> if failures and strict mode: raise [DQ_VALIDATION_FAILED]
```

7.6 DQ Operational Modes

- warn mode (dq_fail_on_issues=false): logs issues and completes
- strict mode (dq_fail_on_issues=true): logs issues and fails DqEngine task

This is intentional behavior, not a platform failure.

8. Source Data Simulator (scripts/00_mongo_simulator.py)

Modes:

- reset_and_seed: recreates clean baseline dataset
- add_good: inserts valid incremental records
- add_bad: inserts records designed to fail DQ rules

Flow:

```
parse --mode and mongo uri
  -> connect to ecommerce.customers
  -> execute selected operation
  -> close client
```

Purpose:

- deterministic testability of Bronze/Silver/DQ behaviors
- reproducible defect scenarios for the AI agent

9. App Design (app/agent_app.py)

The app is both API and UI.

9.1 Endpoints

- GET / -> redirects to /ui
- GET /ui -> browser interface
- GET /health
- GET /latest-issue
- GET /issues-summary
- POST /analyze
- GET /docs (OpenAPI UI)

9.2 SQL Access Model

The app reads Delta tables through Databricks SQL Warehouse using:

- DATABRICKS_SERVER_HOSTNAME
- DATABRICKS_HTTP_PATH
- DATABRICKS_TOKEN

9.3 Analysis Modes

POST /analyze supports:

- analysis_mode=latest: analyze only latest issue
- analysis_mode=summary: analyze aggregated failures in a window

Additional controls:

- window_hours, max_issues, sample_size
- concise toggle

9.4 LLM Provider Strategy

Provider selected by LLM_PROVIDER.

Implemented strategies:

- Gemini:
- primary SDK: google-genai
- fallback: google-generativeai
- OpenAI: openai SDK
- Anthropic: anthropic SDK

Flow:

```
POST /analyze
  -> fetch issue data (latest or summary window)
  -> build structured prompt
  -> route to selected provider
  -> return provider + issue + summary + analysis text
```

9.5 UI Rendering

The /ui frontend consumes API endpoints and includes:

- mode selector (latest / summary)
- controls for window and sample size
- summary and latest payload preview panels
- markdown-to-HTML rendering for analysis output

The renderer improves readability for headings, lists, and code blocks.

10. Agentic AI Value in this POC

This project uses “agentic AI” as an operational assistant for DQ incidents.

It does not change data automatically. It provides decision support:

- summarizes probable root cause
- points to likely failing stage
- proposes minimal SQL/PySpark fixes
- returns validation checks to close incident loops

Why it matters even with a single job + tasks:

- reduces mean time to diagnose recurring bad records
- standardizes remediation guidance
- improves handoff quality between platform and data engineering roles

11. Configuration and Secrets Handling

11.1 .env and .load-env.ps1

.load-env.ps1 reads .env and sets:

- process env vars (KEY=value)
- bundle vars (BUNDLE_VAR_key=value)

Flow:

```
read .env line by line
  -> ignore comments and empty lines
  -> set process env var
  -> set BUNDLE_VAR_<lowercase_key>
```

11.2 Required Inputs

- MONGODB_URI
- LLM_PROVIDER
- provider API key(s)
- DATABRICKS_* SQL connector keys
- optional DQ_FAIL_ON_ISSUES

12. Design Patterns Used

1. Metadata-Driven Configuration Pattern
2. Medallion Layering (Bronze/Silver)
3. SCD Type 2 Historical Modeling
4. Rule Engine Pattern
5. Strategy-like Provider Routing
6. Fail-Fast / Fail-Open Toggle
   - ingestion behavior controlled by metadata tables
   - raw-normalized capture then curated SCD2
   - full historical lineage of customer changes
   - externalized SQL predicates in dq_rules
   - runtime LLM provider selection with provider-specific implementation
   - configurable strict or warning mode for DQ enforcement

13. Operational Runbook (Condensed)

```
1) Load env
   . .\.load-env.ps1

2) Validate and deploy
   databricks bundle validate -t dev
   $env:BUNDLE_VAR_dq_fail_on_issues="false"
   databricks bundle deploy -t dev

3) Seed data
   python scripts/00_mongo_simulator.py --mode reset_and_seed

4) Run job
   databricks bundle run -t dev poc_pipeline_job

5) Start app
   databricks bundle run -t dev dq_rescue_agent
```

14. Known Constraints and Tradeoffs

- bundle name still includes scala token (agentic_dq_scala_poc) despite Python runtime migration
- DQ rules are currently table-driven but domain scope is single-table in this POC
- AI output is advisory, not auto-remediation
- data source reader currently optimized for Mongo simulation path

15. Extension Paths

1. Multi-source metadata rows with generic source adapters
2. Rule severities with differentiated actions (warn/error/quarantine)
3. Quarantine table for bad records instead of only issue logs
4. Automated incident ticket payload generation from /analyze
5. Unit/integration test harness for each pipeline step and API endpoint
6. Rename bundle identifier to remove legacy scala naming

16. Summary

This POC demonstrates a pragmatic metadata-driven data platform pattern in Databricks Free Edition:

- source metadata controls ingestion
- SCD2 creates auditable Silver history
- table-driven DQ rules produce explicit issue logs
- configurable strict/warn behavior supports multiple operating modes
- an agentic AI app accelerates diagnosis and remediation planning

The result is a compact but production-aligned reference architecture for data quality operations.